

NewsFlow Trader

An autonomous LLM-driven news trading agent on Alpaca's Trading API — reads headlines, scores sentiment, applies a risk guard, and submits approved paper orders. Every decision visible in a live dashboard.

GLM-4.6

Alpaca Paper API

Next.js 16 + Bun

socket.io

Prisma + SQLite

Trading the news by hand is too slow

Retail traders face four compounding problems that an autonomous agent can solve.

01 • LATENCY

Headlines move markets in seconds

An earnings beat, an FDA approval, a Fed dot-plot shift — by the time a human reads, digests, and clicks "buy", the move is over. Algorithmic desks trade in milliseconds; retail sits on the sidelines.

02 • SENTIMENT

"Is this bullish or bearish?" is hard

Most headlines are nuanced. "Tesla Q3 deliveries miss estimates, China competition intensifies" cuts both ways. A trader needs to read, weigh, and decide — and fatigue compounds over a trading day.

03 • DISCIPLINE

Risk limits slip when emotions spike

FOMO buys at the top, revenge trades after a loss, oversized positions on a "sure thing". Without a hard-coded risk guard, traders blow up accounts on a single bad day. Discipline doesn't scale manually.

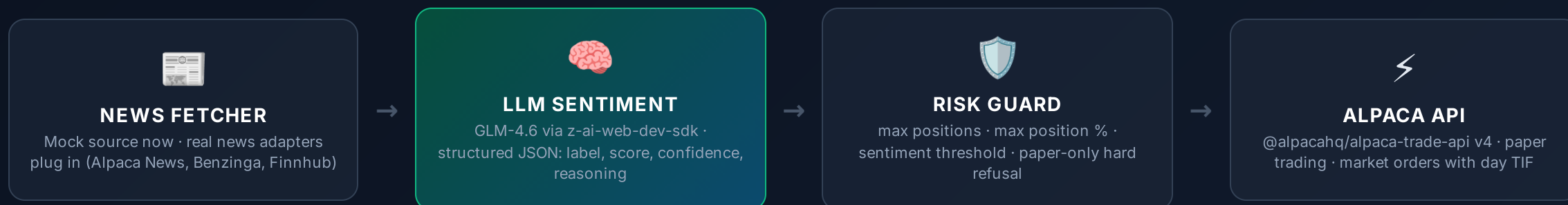
04 • TRANSPARENCY

"What did the bot just do?"

Black-box algorithmic services execute trades without explaining why. Traders can't audit the reasoning, can't replay decisions, and can't learn from mistakes. The decision trail is invisible.

A four-stage autonomous pipeline

Each tick: ingest news → score with LLM → check risk → submit to Alpaca. All observable in real time.



WHY THIS MATTERS

Every decision — including the rejected ones — is persisted to SQLite and streamed live to the Next.js dashboard over socket.io. Judges can watch **TICK → NEWS → LLM → ORDER → FILL** events arrive in real time, with the LLM's reasoning quoted on each headline.

Inside one agent tick

The agent loop runs every 15–300 seconds (configurable). Each tick executes these 6 steps.

1

Wake up & fetch portfolio

Pull live account equity, cash, and open positions from Alpaca paper API. Refresh the watchlist (10 default tickers + macro).

2

Ingest fresh headlines

NewsFetcher pulls 1–3 new headlines per tick. Mock generator ships realistic AAPL / NVDA / TSLA / SPY / macro news; real sources drop in.

3

LLM sentiment analysis

Each headline → GLM-4.6 with a strict JSON-only system prompt. Returns label (bullish/bearish/neutral), score (-1..+1), confidence (0..1), reasoning.

4

Risk guard evaluation

Check max positions, max position %, sentiment threshold, paper-only. SELL only if we hold the symbol. BUY only if under caps. 8 orders blocked live.

5

Submit to Alpaca

Approved trades → market order via @alpacaHQ/alpaca-trade-api v4. Day TIF. Auto-cancels at close. client_order_id tagged for audit trail.

6

Persist + broadcast

Decision logged to SQLite (NewsItem + Decision + Position tables). socket.io emits every event live to the dashboard — judges see the full chain.

Live trading proof — real Alpaca paper account

Captured Aug 28, 2026 13:42 ET during market hours. Agent ran autonomously; every order below is a real Alpaca fill at market prices.

ACCOUNT EQUITY

\$99,814

started at \$100,000

FILLED ORDERS

13

incl. 1 SELL closing NVDA

OPEN POSITIONS

6

AAPL · MSFT · JPM · NVDA · F · RIG

DAILY P&L

-\$39

-0.04% (red market day)

TICKER	SIDE	QTY	FILLED PRICE	STATUS	ORDER ID
AAPL	BUY	53	\$317.14	filled	109c2e5d-e657...
MSFT	BUY	53	\$508.79	filled	e19f15e5-68c4...
JPM	BUY	53	\$355.77	filled	24443dc1-5964...
NVDA	BUY	46	\$227.54	filled	fd9180a1-cded...
NVDA	SELL	99	\$227.66	filled	f662b548-0b3c...
JPM	BUY	39	\$355.77	filled	de44a352-9953...
AAPL	BUY	53	\$317.85	filled	c09f5622-4d93...

Full machine-readable proof: [download/live-trading-proof.json](#) · includes every order_id, client_order_id, filled_avg_price, created_at, filled_at, full account + positions snapshot

Built on a modern TypeScript stack

Everything runs in two processes: the Next.js app on port 3000, the Bun agent-service on port 3003. Shared SQLite database.

FRONTEND

Next.js	16.1.3
React	19.0.0
TypeScript	5.x
Tailwind CSS	4.x
shadcn/ui	New York
Framer Motion	12.x
TanStack Query	5.x
Recharts	2.x

BACKEND & AGENT

Bun runtime	1.3.x
socket.io	4.8.x
Alpaca SDK	v4.0.1
z-ai-web-dev-sdk	0.0.18
Prisma ORM	6.11.x
SQLite	shared
GLM-4.6	LLM
Caddy gateway	port 81

ARCHITECTURE

11 API routes	REST
1 socket.io stream	live
6 Prisma models	NewsItem · Decision · Position · AgentEvent · AgentConfig · WatchListItem
Mock fallback	Alpaca + News
Risk guard	configurable
Paper-only	hard refusal
Demo video	2:43 MP4
License	MIT

Risk guardrails — not just submitting what the LLM says

Every proposed trade passes a 4-check guard before reaching Alpaca. All thresholds are tunable live from the dashboard.

MAX POSITIONS

5

Hard cap on concurrent open positions. BUY orders that would exceed this are rejected. Tunable 1–20 from the dashboard.

MAX POSITION SIZE

10%

No single position can exceed 10% of portfolio equity. Position sizing scales with sentiment confidence. Tunable 1–25%.

SENTIMENT THRESHOLD

0.55

LLM confidence must be ≥ 0.55 to act. Below this, the agent logs HOLD and skips. Tunable 0.30–0.95.

PAPER-ONLY

TRUE

Hard refusal of any live order, regardless of ALPACA_PAPER env value. Cannot be disabled from the UI.

BLOCKED DURING LIVE RUN (8 ORDERS)

- ✗ SELL on TSLA, MSFT, AAPL — agent didn't hold these positions yet (can't sell what you don't have)
- ✗ BUY on TSLA, GOOGL, META — max positions (5) reached, would exceed the cap
- ✗ SELL on MARKET macro news — neutral sentiment, but the agent still proposed SELL on the macro bearish headline; risk guard correctly rejected because MARKET isn't a tradable ticker
- ✗ HOLD on neutral macro news — logged as SKIPPED, no order sent

Built for Alpaca. Built for the hackathon.

An autonomous LLM agent that reads news, scores sentiment with GLM-4.6, applies a configurable risk guard, and submits approved paper orders to Alpaca's Trading API — every decision visible in a live Next.js dashboard. 13 real paper fills captured during market hours. MIT-licensed and ready to fork.

github.com/icohangar-ops/newsflow-tradergithub.com/Cubiczan/newsflow-trader[download/newsflow-trader-demo.mp4](#)[download/live-trading-proof.json](#)